

WP-8 Background Jobs, Notifications & Integrations

Weeks: Week 7 Track: Backend

Goal. The automation that makes the platform run itself — built as proper background jobs, not a naive timer. This package is as much about *how* a job is engineered (safe to run continuously, safe across multiple instances, observable when it fails) as about what the jobs do.

Tasks — the hosted-service foundation

- ☐ Build the worker as a .NET hosted service (BackgroundService / IHostedService). You own the loop, the timing, and the shutdown — no job library hiding it.
- ☐ Resolve a fresh DI scope per run — a hosted service is a singleton, so never inject scoped services (like your DbContext) directly into it.
- ☐ Honour the CancellationToken for graceful shutdown: a job in flight must stop cleanly when the app is asked to stop, not get killed mid-write.
- ☐ Make the poll interval and batch size configurable, not hardcoded.

Tasks — scale, performance & resilience

- ☐ Process due work in bounded batches, so a backlog of thousands of reminders can't load everything into memory at once.
- ☐ Query only the rows actually due, using the filtered indexes from WP-1 — never scan the whole table each tick.
- ☐ Isolate failure per item: one bad notification must not abort the whole batch. Catch, log, mark it for retry, and move on.
- ☐ Add a retry policy with backoff for transient failures (e.g. the email provider being briefly down), and a cap so a permanently-failing item doesn't retry forever.

Tasks — logging & exception handling inside jobs

- ☐ Flow a correlation ID through each job run (and each item) so a job's Serilog output is traceable end-to-end, exactly like a web request.
- ☐ Log the shape of each run: how many items picked up, succeeded, failed, and how long it took — you can't operate a job you can't see.
- ☐ Make sure an unhandled exception in a job is caught and logged, and never silently kills the hosted service for the rest of the app's lifetime.

Tasks — multi-instance safety (the lock)

- ☐ Assume the app runs as more than one instance. Two copies of the same job must not both process the same due work.
- ☐ Build a service lock so only one instance owns a given job for a run: the owner is identifiable, the lock has a lease that expires, and a heartbeat renews it while the job is alive.
- ☐ Handle the instance dying mid-run: the lease must expire so another instance can safely take over the next run — no job wedged forever because its owner crashed.

Tasks — the jobs & integrations themselves

- ☐ Reminder job: email members a configurable interval before a booking.
- ☐ No-show job: release confirmed bookings after a grace period.
- ☐ Stale-approval job: expire or escalate un-actioned approval requests.
- ☐ Integrate a transactional email provider, going through the retry/backoff path above.
- ☐ Generate an ICS feed a member can subscribe to.

HARD PROBLEM Idempotency. A hosted service has no built-in memory of what already ran — if the app restarts mid-batch, or two instances race, a job WILL run twice. Running twice must never double-send a reminder or double-release a slot. Lean on the database for the guarantee (e.g. a unique constraint that makes a duplicate send physically impossible), not on hoping it doesn't happen. Design for the retry, not the happy path.

HARD PROBLEM The instance lock is a distributed-lock problem — the same family as booking concurrency, now for job ownership. Target: a lease table with an owner id + expiry that a heartbeat renews, and think through the traps (a lease that expires mid-run while the owner is still alive; clock differences between instances; the renew-vs-expire race). If that's out of reach, `sp_getapplock` is an acceptable floor — it auto-releases if the owning connection dies — but be able to say what it does and doesn't protect against versus the lease approach.

OPEN DECISION What precisely constitutes a “no-show,” who or what determines it, and what is the grace period? Decide and document.

Acceptance criteria

- ✓ The worker runs as a hosted service, resolves a scope per run, and shuts down cleanly on cancellation without corrupting in-flight work.
- ✓ Due reminders are sent exactly once — even if the job runs twice or the app restarts mid-batch.
- ✓ With two instances running, a given due item is processed by exactly one of them, and a crashed owner's lock is taken over on the next run.
- ✓ One failing item is logged and retried without aborting the rest of the batch; a permanently-failing item stops retrying after a cap.
- ✓ Each job run emits structured logs (counts, duration, correlation id) that let you see what it did.
- ✓ No-show bookings are released and the slot becomes bookable again.
- ✓ Confirmation, rejection, cancellation, and reminder emails are sent on the right events.
- ✓ A confirmed booking is subscribable as an ICS feed.

WP-9 AI-Assisted Booking

Weeks: Weeks 7–8 Track: Both

Goal. A small, genuinely useful AI feature that removes friction from booking — while the booking engine stays the source of truth.

Tasks

- ☐ Pick one: natural-language booking (“book a lab for 3 next Tuesday afternoon”) or smart-slot suggestion.
- ☐ Integrate an LLM API from the backend.
- ☐ Translate the AI's proposal into a concrete, candidate booking.
- ☐ Validate every AI proposal against the same rules as a manual booking before anything is confirmed.
- ☐ Require explicit user confirmation; never auto-finalize.
- ☐ Build the UI for the feature.

HARD PROBLEM The AI proposes; it must never bypass the booking engine. Every suggestion runs through the same validation and concurrency guarantees as a manual booking. Treat AI output as untrusted input.

Acceptance criteria

- ✓ A user reaches a valid, confirmable booking via the AI feature.
- ✓ An AI proposal for an unavailable or rule-violating slot is caught and never confirmed.
- ✓ The user always confirms explicitly.

